

Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación
CC3067 — Redes



Excelencia que trasciende

DEL VALLE
GRUPO EDUCATIVO

Laboratorio 2

Esquemas de detección y corrección de errores

Hugo Javier Cifuentes Carredano — 23079

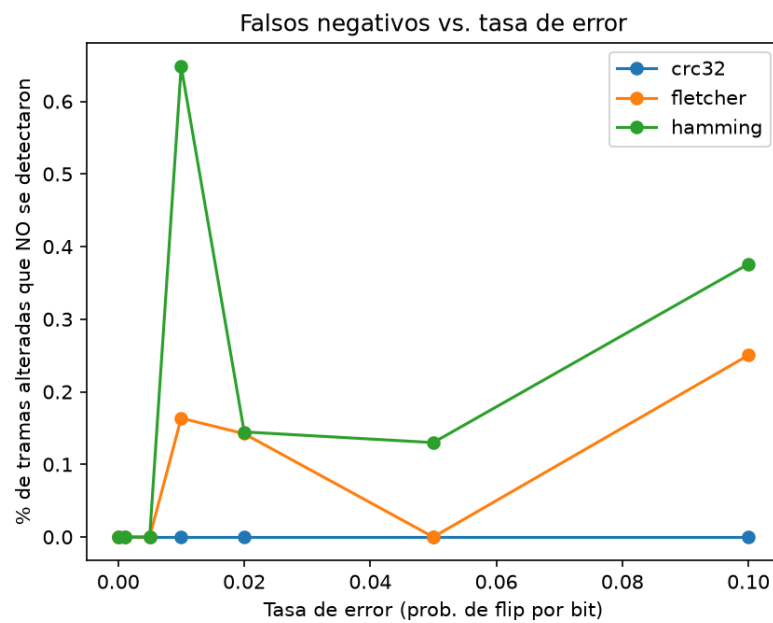
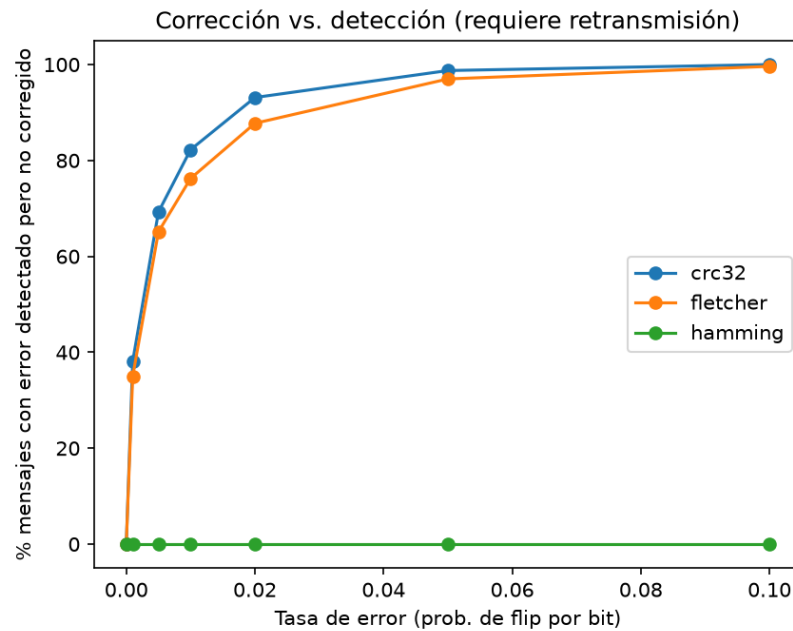
Brandon Werner Rivera Cabrera — 23088

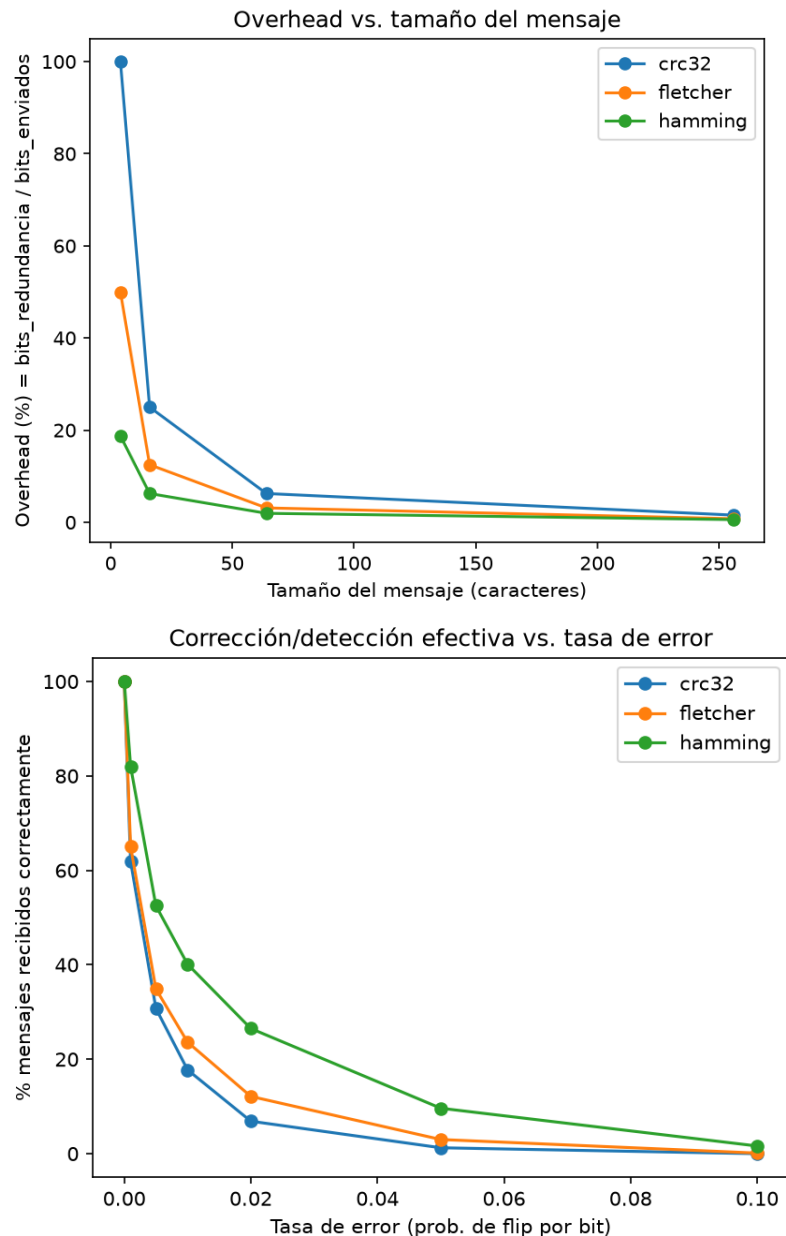
Guatemala, agosto de 2026

1. Repositorio Github

<https://github.com/Brariv/lab2-redes>

2. Benchmark





3. Descripción de la práctica

El laboratorio consistió en implementar una arquitectura de capas (aplicación, presentación, enlace, ruido y transmisión) para simular la comunicación entre un cajero automático (emisor) y el servidor de una entidad financiera (receptor) a través de sockets TCP, sobre un canal que introduce errores de bit con una probabilidad configurable.

Cumpliendo con el requisito de que emisor y receptor estén en lenguajes distintos, el cajero (emisor) se implementó en JavaScript (Node.js) y el servidor bancario (receptor) en Python. Ambos lados comparten únicamente el formato de trama que viaja por el socket: una línea de encabezado JSON (algoritmo, tamaño de bloque, cantidad de bits) seguida de la trama de bits en

sí, cada una delimitada por salto de línea. Esto permite que cada lado lea la trama de forma independiente sin depender de un framing binario más complejo.

Se implementaron los dos algoritmos sugeridos por el enunciado, uno por integrante:

- Corrección de errores — Código de Hamming, con cálculo dinámico de bits de paridad r según la relación $m + r + 1 \leq 2^r$, inserción de paridad en posiciones potencia de 2, y corrección de un bit mediante síndrome.
- Detección de errores — se implementaron ambas alternativas propuestas por el enunciado (Fletcher checksum, configurable en bloques de 8/16/32 bits con padding, y CRC-32 con el polinomio estándar de 32 bits), en lugar de una sola.

La capa de ruido, que solo existe del lado del emisor (tal como indica el diagrama de arquitectura del enunciado), aplica bit-flips independientes según una tasa de error solicitada al usuario en cada envío, incluyendo los bits de redundancia.

4. Resultados

4.1 Corrección de un bug crítico en la ruta de éxito de Hamming

Durante las pruebas se detectó un bug en `capas/enlace.py` (línea 32): en el camino sin error de Hamming, `verificar_integridad` devolvía el codeword completo (incluyendo los bits de paridad) en lugar del mensaje decodificado, por lo que el texto llegaba corrupto incluso con una tasa de error de 0. El bug fue corregido y verificado con una prueba de caso simple ("Hi" → "Hi") antes de correr la batería completa de pruebas.

4.2 Estado de la integración emisor (JS) – receptor (Python)

La integración funcional completa (arquitectura de capas, framing de trama, codificación/decodificación ASCII, sockets) quedó verificada en ambos lenguajes. Sin embargo, las implementaciones de los algoritmos del lado del emisor (JavaScript) presentan bugs que las hacen incompatibles con el receptor (Python) en la práctica:

- Hamming (JS): nunca calcula la paridad ni el síndrome correctamente, por lo que hoy no corrige ningún bit en la integración real emisor–receptor.
- Fletcher (JS): overflow sin aplicar el módulo correspondiente en la suma, generando checksums que no coinciden con los calculados del lado Python.
- CRC-32 (JS): implementa un algoritmo distinto al usado en Python, por lo que los residuos no son comparables entre los dos lados.

Dado que las pruebas cuantitativas de la sección 4.3 requerían mediciones consistentes y aisladas por algoritmo, la batería de pruebas se corrió sobre las implementaciones Python de los tres algoritmos (que sí están verificadas y funcionan correctamente), simulando el par codificación → ruido → verificación → corrección dentro del mismo proceso. Esto aísla el comportamiento de cada algoritmo del bug de integración JS descrito arriba, el cual se documenta como limitación conocida en la sección 5.

4.3 Batería de pruebas

Se implementó pruebas/benchmark.py con las funciones ejecutar_prueba, correr_bateria (exporta a CSV sin depender de pandas) y graficar_resultados (matplotlib), según lo solicitado en el punto c del enunciado. Se ejecutó una corrida de 16,800 pruebas, variando 3 algoritmos $\times$ 4 tamaños de mensaje $\times$ 7 tasas de error $\times$ 200 repeticiones. Los resultados completos (CSV y 4 gráficas PNG) se incluyen en el repositorio.

Tabla 1. Porcentaje de mensajes recibidos correctamente, por tasa de error de bit y algoritmo.

Tasa de error	Hamming	Fletcher	CRC-32
0.001	82.0%	65.0%	61.9%
0.01	40.1%	23.6%	17.8%
0.05	9.6%	3.0%	1.2%

Tabla 2. Overhead promedio (bits de redundancia / bits totales) por tamaño de mensaje.

Tamaño (caracteres)	Hamming	Fletcher	CRC-32
4	18.75%	50.0%	100.0%
256	0.59%	0.78%	1.56%

Corrección vs. detección: en la gráfica de mensajes "irrecuperables" (error detectado pero no corregible), Hamming se mantiene en 0% en todo el rango de tasas de error probado, mientras que Fletcher y CRC-32 suben hasta cerca de 100% de mensajes irreuperables a tasas de error altas. Esto no significa que Hamming sea perfecto: como se discute en la sección 5, el 0% es engañoso, porque Hamming siempre reporta haber corregido el mensaje, incluso cuando la corrección fue incorrecta.

5. Discusión

¿Qué algoritmo tuvo mejor funcionamiento? En términos de porcentaje de mensajes recibidos correctamente, Hamming fue consistentemente superior a Fletcher y CRC-32 en las tres tasas de error probadas (Tabla 1), lo cual es esperado: al ser capaz de corregir un bit erróneo sin necesidad de retransmisión, recupera mensajes que los otros dos algoritmos simplemente descartan como fallidos. CRC-32 fue el más frágil de los tres pese a tener el mayor overhead, ya que solo detecta errores y con tasas de error altas la probabilidad de que un mensaje largo tenga dos o más bits alterados crece rápidamente.

¿Qué algoritmo es más flexible para aceptar mayores tasas de error? Hamming, por la misma razón: al corregir errores de un bit en lugar de solo detectarlos, degrada de forma más gradual conforme sube la tasa de error (Tabla 1), mientras que Fletcher y CRC-32 pierden efectividad más rápido porque cualquier error detectado implica descartar el mensaje.

¿Cuándo conviene detección en lugar de corrección? La ventaja de corrección (Hamming) tiene un costo oculto importante que se observó en las pruebas: el algoritmo implementado corrige correctamente un bit de error, pero no tiene forma de distinguir un mensaje con un solo bit erróneo

de uno con dos o más bits erróneos — en este último caso, aplica una "corrección" igualmente, solo que incorrecta, sin señal de que el mensaje sigue siendo inválido. Esto se refleja en que la tasa de mensajes "irrecuperables" de Hamming se mantiene en 0% en toda la corrida (sección 4.3): el algoritmo nunca admite que no pudo corregir, aunque el contenido final termine siendo erróneo. Fletcher y CRC-32, en cambio, sí reportan honestamente cuándo un mensaje no pudo verificarse, permitiendo solicitar una retransmisión. Por esto, detección pura conviene en escenarios donde retransmitir es barato y la integridad del dato importa más que la latencia (por ejemplo, transferencias bancarias, donde un monto corregido incorrectamente por Hamming sería mucho más peligroso que un mensaje descartado y reenviado); corrección conviene cuando la retransmisión es costosa o imposible (comunicación en tiempo real, enlaces de alta latencia) y una probabilidad baja de corrección silenciosa incorrecta es un riesgo aceptable frente al beneficio de recuperar el mensaje sin una vuelta adicional.

Overhead: como muestra la Tabla 2, la elección de algoritmo importa mucho más en mensajes cortos. CRC-32 añade 32 bits fijos de redundancia sin importar el tamaño del mensaje, lo que en un mensaje de 4 caracteres (32 bits de datos) representa 100% de overhead; Hamming, al escalar sus bits de paridad logarítmicamente con el tamaño del mensaje, es notablemente más eficiente en mensajes cortos. En mensajes largos (256 caracteres) las tres alternativas convergen a un overhead bajo (<2%), por lo que en ese régimen la elección debería basarse más en la capacidad de corrección que en el overhead.

Limitación de integración conocida: el hallazgo de que las implementaciones JavaScript de los tres algoritmos (Hamming, Fletcher y CRC-32) tienen bugs que las hacen incompatibles con las implementaciones Python del receptor (sección 4.2) es en sí mismo relevante para la discusión del enunciado: ilustra por qué el formato de trama y el algoritmo de integridad deben especificarse de forma completamente inequívoca (incluyendo detalles de implementación como el manejo de overflow en Fletcher o el polinomio y la convención de bit-order en CRC-32) cuando emisor y receptor están en plataformas distintas, ya que una discrepancia silenciosa en la implementación del algoritmo de integridad es indistinguible, desde la capa de enlace, de un error genuino introducido por el canal ruidoso.

6. Comentario grupal sobre el tema

Nos pareció interesante que, en la práctica, la falla más costosa del laboratorio no vino del canal ruidoso que estábamos simulando, sino de una discrepancia de implementación entre los dos lenguajes usados para emisor y receptor. Esto refuerza una idea central de la capa de enlace: que la confiabilidad de un protocolo depende tanto de que el algoritmo esté matemáticamente bien definido, como de que ambos extremos lo implementen de forma idéntica bit a bit.

7. Conclusiones

- Hamming ofreció el mejor porcentaje de mensajes recibidos correctamente en las tres tasas de error probadas, gracias a su capacidad de corregir un bit sin retransmisión, pero esa misma capacidad de corrección introduce un riesgo silencioso: con dos o más bits erróneos, el algoritmo aplica una corrección incorrecta sin reportarlo como fallo.

- CRC-32 fue el algoritmo más frágil frente a tasas de error altas y, junto con Fletcher, resultó considerablemente más costoso en overhead para mensajes cortos, aunque ambos tienen la ventaja de reportar honestamente cuándo un mensaje no pudo verificarse.
- La elección entre corrección y detección pura no debe basarse únicamente en el porcentaje de éxito bruto, sino en el costo relativo de una retransmisión frente al riesgo de aceptar silenciosamente un dato corrupto — relevante en particular para el caso de uso bancario simulado en este laboratorio.
- Se corrigió un bug crítico en la ruta de éxito de la verificación de integridad (capas/enlace.py) que corrompía todo mensaje Hamming incluso sin ruido, evidenciando la importancia de probar primero el caso de tasa de error 0 antes de introducir ruido.
- Persisten bugs de integración específicos de la implementación JavaScript del emisor en los tres algoritmos, que impiden hoy una comunicación end-to-end correcta entre el cajero (JS) y el servidor bancario (Python); las mediciones cuantitativas de este reporte se realizaron sobre las implementaciones Python, ya verificadas.

8. Citas y referencias

Hamming, R. W. (1950). Error detecting and error correcting codes. *The Bell System Technical Journal*, 29(2), 147–160.

Fletcher, J. G. (1982). An arithmetic checksum for serial transmissions. *IEEE Transactions on Communications*, 30(1), 247–252.

International Organization for Standardization. (1993). ISO/IEC 3309: Information technology — Telecommunications and information exchange between systems — High-level data link control (HDLC) procedures — Frame structure (definición del polinomio estándar CRC-32).

Universidad del Valle de Guatemala, Departamento de Ciencias de la Computación. (2026). CC3067 Redes — Enunciado de Laboratorio 2: Esquemas de detección y corrección de errores.